OpenNVR Agentic Workflows

A **Workflows** page where a site builds automations from a short vertical flow: a trigger, a few steps, an outcome. One step kind, **AI**, is where the judgement lives: it looks at a picture or investigates with tools, and returns a typed decision the flow branches on. **Site rules**, kept in one place, make every AI step accurate for this site. People and plain rules move things; AI informs and recommends. Everything runs on the box.

Status final, peer-reviewed twice; corrections in section 13

Replaces the three earlier drafts; the research behind it is here **Repository** open-nvr

WORKFLOW

The skeleton: trigger, steps, outcome.

Three to seven steps, read top to bottom. Deterministic, versioned, every run recorded. Drawn, written as YAML, or drafted from a sentence.

AI STEP

The judgement: Look, or Investigate.

Look answers one question from a picture or text. *Investigate* works toward a goal with tools. Both return a decision from a list, fields, a reason and a confidence.

SITE RULES

The knowledge: facts about this site.

"Cleaners come at 5 am." "Deliveries use the back gate." Written once, read by every AI step. Where most accuracy fixes go.

1 Review of the research design

The research design was checked against the code and against the question "would a site owner understand this in one sitting?". Six changes came out of it.

FINDING	CHANGE IN THIS PLAN
Two AI step kinds (Ask AI and Agent) with different panels is one concept too many for users.	One **AI step** with a mode switch: *Look* (one input, no tools) or *Investigate* (goal plus tools). Same panel, same output shape.
The tool list promised things that do not exist: speakers, a staff schedule, a switch-port check.	Tools are limited to what exists or is cheap. The camera agent already has ten tools (snapshot, describe, detect, recent events and plates, faces, footage and history search). Speakers and Call move to phase 3 behind an app. Schedules and lists live on the Site rules page, which is new.
Where the agent loop runs was unclear: the camera agent is an optional container, the workflow engine is in core.	The loop, the LLM clients and schema-constrained output move into the app SDK as a library. Core's engine and the camera agent both use it. The model (Ollama) stays in the camera-agent overlay; without it, AI steps are shown as unavailable and rule-based workflows still run.
Four questions were left open.	Closed in section 12: workflows lead and a chat message is just a trigger; tool modes are a site policy a step may tighten but never loosen; six templates named; Call stays in phase 3 but its contract is in the schema from phase 1.
"Agent-decided alerts capped at high" conflicted with the earlier medium cap.	One rule: an AI-decided alert is capped at *medium* until the step has a recorded accuracy of 80% over at least 20 marked runs, then *high*. *Critical*, which triggers phone calls through alarm actions, always needs a person or a plain rule.
Limits, chain depth, Test mode, approvals as separate records and camera ids were right in the simplified plan and were missing from the research page.	All carried into this plan.

2 The product

Workflows is a new item in the left pane, in the pinned NVR group after Cameras. A workflow has one trigger, then steps, then outcomes. AI, If, Ask a person and Call split the flow into one path per outcome. Paths never loop back. An *Always* section runs after whichever path ran.

STEP	WHAT IT DOES	OUTCOMES	COMES FROM
`Trigger`	An app's alert; a contracted event such as a gate decision or plate read; a camera going offline; a schedule; a button; a message to the camera agent (phase 2). "A person in a zone for N seconds" is the intrusion or loitering app's alert in phase 1; a native zone trigger is a phase 2 item of its own.	fires	apps, detector, core
`AI`	*Look*: a prompt over a picture, frames or text. *Investigate*: a goal pursued with tools. Section 3.	one per decision, plus `unsure`	core, local models
`If`	A plain rule: time window, site mode, camera, a value from an earlier step, membership of a list.	true · false	core
`For each`	Repeats the steps inside it for each item: cameras in a group, plates in a list, events in a window.	done	core
`Ask a person`	A question with reply buttons to named people or a role; waits; default on timeout.	each reply · no answer	core, inbox and notifications
`Call`	A voice agent phones someone with a goal and script, returns typed outcomes and a transcript. Phase 3.	each outcome · no answer	core, Twilio or SIP
`Action`	Alert, notify, log, save clip, PTZ preset, webhook (to admin-approved destinations only, through the same per-destination policy apps use), Home Assistant, site mode, an app's action such as open the gate, wait.	done	core, apps

States. *Off*, *Test* (everything runs, actions are recorded as "would have"), *On*. New workflows start in Test. A workflow whose step lost its provider shows *Needs attention* and stops.

Limits on every workflow, with defaults: cooldown 1 minute per camera, one run at a time per camera (or per plate or track), 30 runs an hour, 60 AI calls an hour, chain depth 3. Repeat triggers while a run waits for a person join it as "seen again".

Three ways to author. Drag steps in the builder; write YAML; or type a sentence and let the camera agent draft the steps for review. Templates are YAML files in the repository with prerequisites the editor checks.

3 The AI step

One step, two modes, one output shape. The output shape is what makes it composable: the flow branches on the decision, later steps use the fields, and people read the reason.

Look

- **Prompt.** Plain language; click to insert `{plate}`, `{camera}` and other values from earlier steps.
- **Input.** The event's details, plus the camera picture now, a few seconds of frames (for movement), or text such as an intercom transcript.
- **Runs** the site's vision model through KAI-C for pictures, the local language model for text. One call, usually one to three seconds.
- **Use it** for most cases: is someone at the barrier, is this a delivery, is the door open, helmet and vest.

Investigate

- **Goal.** One sentence the agent works toward.
- **Instructions** for this step; Site rules are added automatically.
- **Tools** picked from section 4, each with its mode. The model chooses which to use and in what order, up to the limits.
- **Memory.** This run (default), or this camera's recent decisions, read-only.
- **Limits.** Tool calls and seconds, with defaults set by hardware class the way the installer sizes models: 6 calls and 45 s on a GPU box, 4 calls and 120 s on CPU. Investigate is offered on a site only when its language model passes the Investigate cases in the camera agent's eval harness; Look is offered everywhere.
- **Use it** when one picture is not enough: who is this person, has this car been here, what happened before, why is the camera down.

Output, in both modes

- **Decision**: one of the options the author lists (yes / no; delivery / cab / visitor / unknown; ignore / watch / alert). The model is held to the list by schema-constrained output. Below the confidence floor the decision is `unsure`, which always has its own path.
- **Fields**, optional and named by the author: colour, company, who, flat. Available to later steps as `{colour}`.
- **Reason**: one line, shown to people and used in messages.
- **Confidence**: 0 to 1.

If the model is down, slow or over budget, the step answers `unsure` and the run continues. An AI step never crashes a workflow.

4 Tools

Tools are what Investigate can do. They are the camera agent's existing tools plus a few from core, each with a description written for a model and a mode set by site policy. Installing an app or model that declares tools adds them.

TOOL	WHAT IT DOES	DEFAULT MODE	EXISTS TODAY
camera_snapshot, describe_camera	A fresh picture, described by the vision model or answered against a question	auto	yes, camera agent
describe_event, describe_window	Describe a stored event's best frame, or a time window on a camera	auto	yes
detect_objects	What and how many a camera sees now	auto	yes
recent_events, recent_plates	What happened on this or nearby cameras in the last N minutes	auto	yes
recognize_faces	Match against the face gallery	auto	yes; carries the face data scope
search_footage, search_history	Similar scenes or the same person across cameras; past events by text	auto	yes, semantic search
site_lookup	Read a Site rules list: expected visitors, contractors, staff hours, watchlist	auto	new, with the Site rules page
camera_health	Online state, last frame time, stream errors; a ping to the camera's address	auto	partly; ping is new
ask_person	A question with reply buttons; the	ask, by nature	new, on the approvals

TOOL	WHAT IT DOES	DEFAULT MODE	EXISTS TODAY
	run waits		record
notify, alert	A message or an inbox alert, up to the severity cap	auto	yes
save_clip, ptz_preset	Protect footage; point a PTZ camera at a preset	auto	yes
speak, call	Play a message on a speaker; phone someone	ask	phase 3, speaker app and voice stack
open_gate, unlock, set_site_mode, siren	Moves things	not available to AI	as Actions only

Modes are a site policy: an admin sets each tool to *auto*, *ask* or *not available* once, on the Site rules page. A workflow may make a tool stricter for one step (auto to ask) but never looser. An app action declared with `confirm=True` arrives as "not available to AI" and can only be placed as an Action after an Ask a person or an If on the same path.

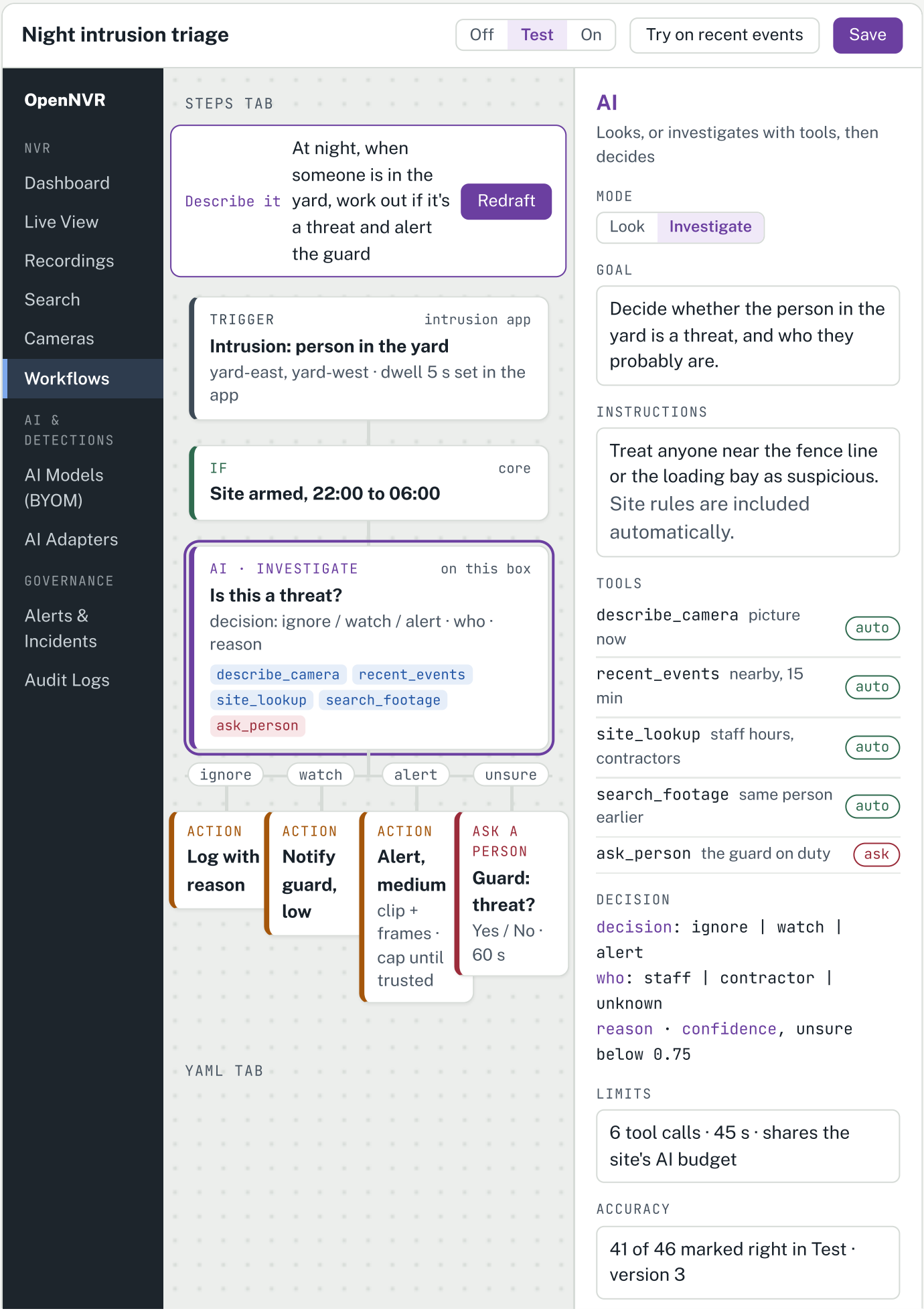
5 Views

5.1 Workflows list

Workflows From a template | Describe one | New

WORKFLOW	STATE	TRIGGER	LAST RUN	TODAY
Night intrusion triage Owner: Priya · Investigate	Test	Person in yard · 2 cameras	02:14 · would have alerted	would have acted 2 times
Gate concierge Owner: Priya · moves things	On	Unknown plate · gate-in	01:52 · approved by Ravi	3 runs
Camera offline triage Owner: Facilities	On	Camera offline 2 min	Yesterday 16:40	0 runs
Morning site report Owner: Sam	On	Every day 07:00	07:00	1 run
Barrier fault to maintenance Owner: Facilities · rules only	Needs attention	Barrier fault · gate app	—	gate app updated; "camera" input changed

5.2 Builder, with an AI step selected



```yaml
name: Night intrusion triage
state: test
limits: { cooldown: 2m, one_at_a_time:
track, per_hour: 20, ai_per_hour: 40 }
when:
  alert.intrusion-detection.intrusion: {
cameras: [yard-east, yard-west] }   #
zone and dwell set in the app
if:
  - site_mode: [armed_home, armed_away]
  - time: { from: "22:00", to: "06:00" }
then:
  - ai:
      mode: investigate
      goal: Decide whether the person in
the yard is a threat, and who they
probably are.
      instructions: Treat anyone near the
fence line or the loading bay as
suspicious.
      tools: [describe_camera,
recent_events, site_lookup,
search_footage, ask_person]
      decision: [ignore, watch, alert]
      fields: { who: [staff, contractor,
unknown] }
      sure: 0.75
      limits: { tool_calls: 6, seconds:
45 }
      ignore: [ log: "{reason}" ]
      watch:  [ notify: { who:
role:guard, severity: low, message: "
{reason}" } ]
      alert:  [ alert: { severity:
medium, title: "Intruder on {camera}",
attach: [clip, frames] } ]
      unsure:
        - ask_person: { who: role:guard,
message: "Possible intruder on {camera}.
Threat?", replies: [Yes, No], timeout:
60s }
```

Mockup. The Steps and YAML tabs work and show the same workflow. Tools marked *ask* pause the run for a person. Nothing that moves things is on the tool list.

5.3 A run, as the Runs view shows it

Night intrusion triage · 02:14:07 Test

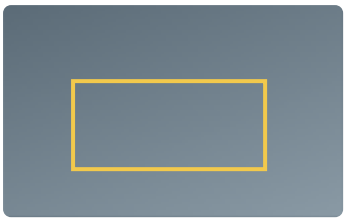
Mark right | Mark wrong

- **Intrusion: person in the yard** `02:14:07`
  yard-east · intrusion app · track 8812 · 6 s in zone
- **Site armed, 22:00 to 06:00** `+0.0 s`
  true
- **describe_camera** `+2.1 s`
  "One adult walking along the fence line toward the loading bay, carrying a long object, no high-visibility clothing."
- **site_lookup: staff hours** `+2.2 s`
  Nobody expected on site. Last contractor sign-out 19:40.
- **recent_events** `+2.4 s`
  Loitering alert at the back lane, 01:58, same direction.
- **search_footage** `+5.9 s`
  Same person at the back lane camera at 01:57 and 02:03.
- **Decision: alert** `+7.0 s`
  who: unknown · confidence 0.88 · "Unknown adult approaching the loading bay with an object at 2 am, seen earlier at the back lane; nobody is expected on site." · version 3
- **Alert, medium** `+7.1 s`
  would have alerted (Test) · clip and 3 frames attached · 4 tool calls, 6.9 s on a GPU box, within limits

5.4 Approvals in Alerts & Incidents

An Ask a person step creates an *approval*, a separate record from an alert, pinned above the alerts. "Acknowledge all" does not touch it. Only people in the step's audience who hold `workflows.approve` and can see the camera get the buttons. Repeat sightings join the open approval.

Alerts & Incidents Acknowledge all alerts



Let in KA01AB1234, a delivery?

Gate concierge · gate-in · 01:52

White van with a courier logo, parcels visible (AI, 0.92). Plate not registered; not seen in 30 days. No visitor expected for this plate.

Let in | Refuse

`72 s left, then refused`

Seen 2 more times while waiting · only operators can answer

ALERTS

Loitering · shopfront-2	medium · 01:40
Unknown visitor · front-door	low · 00:12
Barrier fault · gate-out	high · 23:58

5.5 Site rules

One page under Workflows. Three parts: facts in plain language, lists the tools can read, and the tool policy. Every AI step includes the facts, so they are capped at about 300 words and the page shows the count; longer material goes in lists, which tools read on demand. `site_lookup` reads the lists.

Site rules Save

FACTS

- Staff may be on site until 23:00 on weekdays.
- Cleaners arrive at 05:00, Monday to Saturday, in a white Maruti van.
- Deliveries use the back gate; the front gate is residents only.
- Night shift is 22:00 to 06:00; the guard on duty is in the Guards list.

LISTS

- Expected visitors today (3)
- Contractors this week (2)
- Guards and shifts
- Watchlist plates (5)
- Camera groups: yard, gates, lobby

TOOL POLICY

- Picture, events, search, lookup: **auto**
- recognize_faces: **auto**, face scope declared
- notify, alert: **auto**, medium cap until trusted
- ask_person, speak, call: **ask**
- Gate, locks, siren, site mode: **not available to AI**

5.6 Try on recent events

From the builder, pick up to 20 events from the last days and run the AI step against them. Each shows its decision, reason and trace. Marks made here count toward the step's accuracy. For Investigate, tools run for real but `ask_person` is stubbed with an answer the author picks. The trial runs at low priority with progress and a stop button.

6 Templates

Six ship with the feature, two more in phase 3. Each is named after its pattern and lists what it needs; the editor checks the prerequisites before the workflow can leave Off.

Night intrusion triage

`investigate, then route`

TRIGGER	Intrusion alert from the intrusion app · armed · night
AI	Investigate: picture, nearby events, staff hours, same person earlier → ignore / watch / alert
ACTIONS	Log · notify low · alert (medium until trusted) · unsure asks the guard

Needs: the intrusion app on the yard cameras, a vision model, a model that passes the Investigate harness.

Gate concierge

`look, then approve`

TRIGGER	Gate decision: unknown plate at gate-in
AI	Look: delivery / cab / visitor / unknown, from the picture and today's expected visitors
ASK	Guard: "Let in {plate}, {decision}?" 90 s, then refuse
ACTION	gate-controller open_now on Let in; otherwise notify

Needs: plate app with barrier mode on gate-in; gate-controller app. Checklist: turn off the plate app's own unknown-plate alert for this camera.

Safety gear check

`look, then act`

TRIGGER	Intrusion app: person enters the work zone
AI	Look at 3 frames: helmet and high-visibility vest? yes / no
ACTION	No: notify the supervisor with the frame; count toward the weekly report

Needs: the intrusion app on the zone, a vision model. Speak a reminder is added in phase 3 with a speaker app.

Camera offline triage

`investigate, not vision`

TRIGGER	Camera offline for 2 minutes
AI	Investigate: camera_health, ping, recent offline history, power schedule from Site rules → power / network / camera / unknown
ACTIONS	Webhook to the ticket system with the diagnosis; notify facilities

Needs: nothing beyond core and the local language model.

Morning site report

`for each, then write`

TRIGGER	Every day at 07:00
FOR EACH	Camera group: night events, alerts, offline minutes
AI	Look at the collected text: a one-page report with the three things worth a look, each linked to its clip
ACTION	Email the manager

Needs: the local language model; an email integration.

Investigate on request

`investigate, then verify`

TRIGGER	A message to the camera agent starting "investigate:" (the agent starts the run through a new internal route and polls it for the result)
AI	Investigate: search footage and history, follow the person across cameras, build a timeline with evidence
AI	Look at the timeline: does every claim cite a clip? Drop those that do not
ACTION	Reply in chat with the report; protect the clips

Needs: semantic search enabled, the camera agent, and the agent-to-engine route built in phase 2.

Resident call-back phase 3

`look, then call`

TRIGGER	Visitor at the lobby intercom camera, out of hours
AI	Look at the intercom transcript: which flat?
CALL	Phone the resident: "A visitor for you at the lobby. Let them in?" Let in / Refuse / No answer
ACTION	Unlock on Let in; otherwise speak to the visitor

Needs: intercom audio, the voice stack, a resident list, a speaker app.

Talk-down phase 3

`speak, then watch`

TRIGGER	Night intrusion decided "alert"
ACTION	Speak: "You are on camera; security has been notified"
AI	Look at 30 s of frames: leaving / staying
ACTION	Staying: alert high; Ask a person: call the police?

Needs: a speaker app.

7 YAML

The YAML mirrors the builder. The editor validates it as you type against a schema generated from the step and tool registries. Files live in git if a site wants that, and the SDK's command line applies them. The night triage workflow is in section 5.2; here is the gate concierge, which has an approval and an actuator.

```yaml
name: Gate concierge
state: on
limits: { cooldown: 2m, one_at_a_time: plate, per_hour: 20 }
needs: [app:license-plate-recognition, app:gate-controller, model:vqa]
when:
  event.access.decided: { cameras: [gate-in], reason: unknown }   #
opennvr.events.access.decided.v1
then:
  - ai:
      mode: look
      prompt: Is this vehicle a delivery van, a cab, a private visitor, or unknown? Use logos,
parcels and taxi signs. Expected visitors today are in the site lists.
      input: [picture, site_lookup:expected_visitors]
      decision: [delivery, cab, visitor, unknown]
      sure: 0.8
      delivery: &ask
        - ask_person:
            who: role:operator
            message: "Let in {plate}, a {decision}? {reason}"
            attach: snapshot
            timeout: 90s
            Let in: [ app.gate-controller.open_now: { camera: gate-in } ]   # moves things: a
person decided
            otherwise: [ alert: { severity: medium, title: "Refused at the gate: {plate}" } ]
      cab: *ask
      visitor: *ask
      unknown: *ask
      unsure: *ask
always:
  - save_clip: { camera: gate-in, seconds: 30 }
```

- `when` is one trigger; `if` a list that must all hold; `then` the steps; `always` runs after any path; `needs` the prerequisites; `limits` the run limits.

- An AI step names its decisions as keys, each holding a list of steps. `unsure` always exists.
- Step names say where they come from: `alert` is core, `event.access.decided` a contracted event, `app.gate-controller.open_now` an app action.
- Cameras are written by name and stored as ids, so a rename never breaks a workflow.
- `{plate}` and other values fill messages. They are never inserted into a prompt's instructions; the model receives them as labelled data.

8 Accuracy and safety

> **AI informs and recommends. People and plain rules move things.**
>
> Opening, unlocking, arming and sirens are never on an AI step's tool list. They are Actions that may only follow an Ask a person, a Call, or an If on the same path. An AI-decided alert is capped at medium until the step has 80% accuracy over 20 marked runs, then high. Critical, which triggers phone calls through alarm actions, always needs a person or a plain rule. The cap stops alarm actions, whose default threshold is high; notification channels push at any severity, which is why new workflows start in Test. The builder enforces all of this and offers the fix in one click.

Accuracy

- **Try on recent events** before the workflow runs.
- **Test mode** for every new workflow and every new version of an AI step.
- **Marks and a score.** Right or wrong on each run, by anyone with `workflows.manage` or `workflows.approve`; the score is on the step, on the list and in the go-live dialog.
- **Versions.** Every change to a prompt, goal, tool list or site fact is a version; each run records which decided; roll back is one click.
- **Site rules fix most errors.** A wrong decision that comes from missing knowledge is fixed with one sentence, for every workflow at once.
- **Verify pattern.** A second Look step checks the first's claims against evidence; the investigate template uses it.
- **Describe it** is offered only when the site's language model passes 80% of the drafting cases in the camera agent's eval harness.

Safety

- All AI runs on the box. No picture, plate or face leaves the site.
- Tool modes are a site policy set by an admin; a step can tighten them, never loosen.
- A picture can mislead a vision model, and a sign in view can try to. The design accepts this: a wrong decision costs a wrong alert, never an open gate. Detector-based apps stay on for the same cameras.

- Approvals are their own records with their own permission; acknowledge-all cannot clear them; no answer means the step's default, deny unless changed.
- Limits on every step and workflow; chain depth 3; a workflow ignores alerts it caused itself. The chain travels on workflow alerts; an app that re-alerts drops it until the SDK passes it through in phase 3, so until then the hourly cap and cooldown are what contain a loop through an app.
- Face and plate tools carry the same data scopes apps declare; turning On a workflow that uses them needs an admin; run history follows the recordings retention policy, 30 days by default.
- Test mode performs no action, sends no notification and makes no call; Ask a person, and the `ask_person` tool inside Investigate, record "would have asked" and take the default.

Permissions

PERMISSION	ALLOWS	DEFAULT ROLES
`workflows.view`	See workflows, runs and traces on visible cameras	admin, operator, viewer
`workflows.manage`	Create, edit, test and turn On workflows that neither move things nor read face or plate data	admin, operator
`workflows.approve`	Answer Ask a person steps addressed to them	admin, operator
admin	Edit Site rules and the tool policy; turn On workflows that move things or read face or plate data	admin

9 Architecture

Frontend

`app/src/views/Workflows*.tsx`

- Nav item in `NAV_GROUPS`, keys in `NAV_PERMISSIONS` and `NAV_LABEL_KEYS`, labels in en and fr
- List, builder (Steps and YAML), Runs, Versions, Site rules
- Approval card in `AlertsIncidents.tsx`
- New deps: CodeMirror 6 with YAML, the `yaml` package; Vitest for tests

Core: workflows

`server/workflows/`

- `registry.py`: steps and tools from core, app manifests and model tasks
- `schema.py`, `validator.py`: YAML schema, the safety rule, caps, needs, and camera scope: a workflow may only name cameras its saver can see
- `engine.py`: NATS queue-group consumer, runs persisted step by step, waits as due-time rows, limits, chain depth
- Routers `workflows.py`, `approvals.py`, `site_rules.py`

SDK: agent library

`opennvr_app_sdk/agent/`

- LLM clients (Ollama, OpenAI-compatible) lifted from the camera agent, with schema-constrained output
- The tool loop with traces, limits and typed results
- Tool descriptions for the camera agent's tools
- Used by core's engine and by the camera agent, so both improve together

Models and actions

`existing services`

- Vision through KAI-C and the enrichment governor, below live detection, with a workflow priority at KAI-C so a workflow never takes a camera's budget from an app's plate or face read
- Language model: Ollama from the camera-agent overlay; AI steps unavailable without it
- Alerts on `opennvr.alerts.workflow.*`, so inbox, siren, alarm actions and notifier apply
- App actions through a new internal route that audits workflow, version and owner

How a run works

1. The engine subscribes, as one NATS queue group, to the subjects its On and Test workflows need: `opennvr.alerts.>` and the contracted `opennvr.events.>` subjects; camera connection events come from core's in-process event bus. Schedules are due-time rows. Tier-0 tracks are not consumed in phase 1: zones and dwell stay in the apps that already compute them.
2. A matching event checks the workflow's limits and creates a run row with a correlation id and chain. That id goes on every alert, notification and action the run causes.
3. Each step's inputs, result and timing are written before the next step starts. A restart resumes runs that were waiting. Core runs one process today; the queue group and row-locked claims make a second replica or a separate engine container safe later.
4. An AI step calls the SDK agent library: Look is one schema-constrained call; Investigate is the tool loop with the step's tool list, the site facts, memory and limits. Every tool call is a trace row. Over budget, slow or down means `unsure`.
5. Actions that move things write an intent row, act, then write the result. A crash in between marks the step unknown and alerts, rather than acting twice. They never retry automatically.
6. In Test, Action, Ask a person and Call steps write "would have" rows and perform nothing; AI steps and read tools run for real so the trace is genuine.

Events published while core restarts are missed, as they are for every core consumer today. Making the bus durable with JetStream is a platform decision, not a workflows prerequisite.

10 Data and API

Tables

TABLE	HOLDS
workflows	Name, owner, state, current version, data scopes, needs-attention reason
workflow_versions	YAML as written, parsed document with camera ids, who saved it, note
workflow_runs	Version, mode, trigger event, chain, correlation id, outcome, totals
workflow_run_steps	Step, inputs, result, decision, confidence, frames, timing, mark (right or wrong, by whom)
workflow_tool_calls	Run step, tool, input, result, duration
workflow_approvals	Run, audience, message, evidence, sightings, reply, who, when, expiry
site_rules	Facts (versioned text), lists, tool policy

Endpoints under `/api/v1`

ROUTE	DOES
`GET workflows/registry`	Steps, tools, their schemas and modes
`GET workflows/schema`	YAML schema from the registry
`GET, POST workflows`	List; create
`PUT workflows/{id}`	Save a new version
`POST workflows/{id}/state`	Off, Test, On
`POST workflows/{id}/try`	Run a step or the workflow on chosen events
`GET workflows/{id}/runs`	Runs, steps and tool calls
`POST workflows/runs/{id}/mark`	Right or wrong
`POST workflows/draft`	Describe it
`GET, POST approvals`	Pending; answer
`GET, PUT site-rules`	Facts, lists, tool policy
`POST internal/apps/{id}/actions/{name}`	Engine to app, audited
`POST internal/workflows/{id}/runs · GET …/runs/{run}`	Camera agent starts a run and polls its result (phase 2)

11 Implementation plan

Three phases, each shippable. Estimates assume one backend engineer, one frontend engineer, and the camera-agent owner for the SDK agent library. Phase 1 is deliberately rules-plus-Look so a site gets value before the harder Investigate work lands.

Phase 1

about 9 weeks
Workflows and Look

Backend

- RFC-0004 in `docs/rfcs/` recording the decisions on this page.
- Step registry from core, app manifests (`emits` , `actions` , published events) and model tasks; YAML schema; validator with the safety rule, caps, `needs` , and located errors.
- Engine: queue-group consumer, step-by-step persistence, waits and schedules as due-time rows, limits, chain depth, Test mode, Needs attention. Triggers: alerts, contracted events, zone presence from Tier-0, camera offline, schedule, button.
- Steps: If, For each, Ask a person (approvals record, inbox card, notification with link), Action (alert, notify, log, save clip, PTZ, webhook, site mode, app actions through the internal route), Always.
- AI step, Look mode: SDK LLM clients with schema-constrained output; pictures through KAI-C; the severity cap; try on recent events; marks.
- Site rules: facts and lists, with `site_lookup` readable by Look as input.
- Tables in one Alembic revision; three permissions in `permission_catalog.py` with backfill; audit entries for state changes and approvals.

Frontend

- Nav item; list; builder with Steps and YAML tabs; step forms generated from the registry; Runs with marks; Versions; Site rules page; approval card in Alerts & Incidents. Vitest added to the frontend CI job.

Templates

- Gate concierge, safety gear check, barrier fault to maintenance (rules only), with prerequisites.

Done when a site owner installs Gate concierge from the template, tries the Look step on ten recent gate events, runs it in Test for a day, turns it On, and an operator lets a delivery in from the inbox, with every step visible in Runs.

Phase 2

about 9 weeks
Investigate

SDK and camera agent

- Build a new, small tool loop in `opennvr_app_sdk.agent` for the engine: tool registry with model-facing descriptions, traces, limits, typed results. The camera agent keeps its own loop in phase 2 and switches over in phase 3, once the SDK loop has run on real sites. The two share the LLM clients from phase 1. Named owner: the camera-agent maintainer.
- Tool descriptions for the ten existing tools; new `site_lookup`, `camera_health` with ping, and `ask_person` on the approvals record.
- Drafting cases in the eval harness for Describe it; the `draft_workflow` tool.

Core

- AI step, Investigate mode, on the SDK library: goal, instructions, tools, memory, limits; tool-call traces; tool policy enforcement; accuracy score and the trust gate for severity.
- Tools from apps and models through the registry; the agent-to-engine route and the message trigger; a native zone trigger, estimated on its own; verify pattern.

Frontend

- Investigate panel; tool-call timeline in Runs; accuracy on list and step; go-live dialog; tool policy on Site rules; Describe it.

Templates

- Night intrusion triage, camera offline triage, morning site report, investigate on request.

Done when Night intrusion triage reaches 80% marked accuracy over 20 Test runs on a real site, every run shows its tool calls, and an agent over budget returns unsure without an error.

Phase 3

about 8 weeks
Voice and reach

- Call step: Twilio or SIP line; Whisper and Piper from the camera-agent stack; the local language model holding a scripted conversation with typed outcomes; transcripts on runs; retry once; call recording retention.
- Speaker app with a `speak` action; talk-down and resident call-back templates.
- Approve and refuse inside push messages with signed one-time links.
- App SDK: apps declare tools and alert `fields`; `workflows validate` and `apply` in the SDK command line.
- The engine as its own container for large sites.
- The camera agent switches to the SDK tool loop; chain passthrough in the SDK's `AlertDispatcher`.

Done when the resident call-back runs end to end on a test line, its transcript is on the run, and a third-party app's tools appear in the Investigate panel with no change to OpenNVR.

Testing

- **Validator:** a corpus of workflows that must be refused, including every route from an AI decision to a step that moves things, and every attempt to raise severity above the cap.
- **Engine:** unit tests per step; integration with NATS and Postgres; a test that stops the engine at every step boundary and checks each run completes once and no actuator fires twice; a flood of identical triggers yields one run and one approval; two workflows that trigger each other stop at depth 3.
- **AI step:** schema-constrained output tests against a fake model; unsure on timeout, error and budget; tool policy cannot be loosened by a step.
- **Approvals:** acknowledge-all leaves them; a viewer cannot answer; a user without the camera cannot answer; timeout takes the default.
- **Templates:** each has labelled events and a minimum Look accuracy checked before release; each survives Steps to YAML to Steps unchanged.
- **Load:** 200 cameras of synthetic events with 50 workflows on one core process; event-to-first-step latency and write rate measured.

12 Decisions and risks

DECISION	WHY
Workflows lead; chat is a trigger	Every product studied keeps the agent inside a flow. An investigation started from chat is a workflow whose trigger is a message, so nothing is lost.
One AI step, two modes	One panel to learn. Look covers most cases cheaply; Investigate is there when one picture is not enough.
Tool modes are a site policy	Safety set once by an admin; a step may tighten but never loosen.
Local models only	Sovereignty is the product. Cloud models and Jev are out.
AI informs; people and rules move things	A wrong decision costs a wrong alert, never an open gate.
Severity cap tied to measured accuracy	False positives must not phone people at night; trust is earned with marks, not assumed.
Agent loop in the SDK	Core's engine and the camera agent share one implementation; apps can reuse it.
Engine inside core, replica-safe	Fewer moving parts now; splitting out later is a deployment change.
Approvals as their own records in the inbox	One place to look, with their own permission, uncleared by acknowledge-all.
Site rules as a page	What the no-code products call skills or instructions; where most accuracy fixes go.
Call in phase 3, contract in the schema from phase 1	It is the strongest demo, but it needs the voice stack and a speaker app, and a template written today still loads then.
Six templates first	Three in phase 1 need only Look or rules; three in phase 2 show Investigate on vision and non-vision work.

RISK	ANSWER
Small local models give poor Investigate results on CPU-only boxes	Look works on small models; Investigate and Describe it are offered when the site's model passes the harness; the installer recommends a model size, as it does for the camera agent.
Users expect the AI to open gates	The rule is on the page, in the builder and in the go-live dialog; the one-click fix adds Ask a person. Deterministic triggers still open gates as they do today.
Overlap with the camera agent's alarms, monitors and reports	They keep working. The agent drafts a workflow when a request outgrows an alarm. Migration is a later decision.
Scope grows into a general automation platform	Seven step kinds, no loops, no code, no cloud. Logic beyond that is an app, and the app's alerts, actions and tools appear in workflows.
A noisy trigger floods a site	Required cooldowns, per-hour caps, auto-pause with a notice to the owner, chain depth.
Estimates slip	Each phase is shippable alone; phase 1 is the smallest useful product.

13 Corrections after review

A second review checked the plan against the code after it was posted for community review. These are the changes, all folded into the sections above.

#	FINDING	CHANGE
1	"Person in a zone for N seconds" was listed as a trigger, but zones and dwell exist only inside the intrusion and loitering apps; Tier-0 publishes per-frame tracks with no zones.	Phase 1 uses those apps' alerts as the trigger. A native zone trigger is a separate phase 2 item with its own estimate.
2	The chat trigger and "reply in chat" had no path: the camera agent is a separate container and core cannot push into a conversation.	Phase 2 adds an internal route for the agent to start a run and poll its result. The investigate template depends on it.
3	Lifting the camera agent's 8,000-line loop into the SDK was the riskiest item with the thinnest estimate.	Phase 2 builds a new small loop in the SDK for the engine; the camera agent switches in phase 3 once it has run on real sites. Named owner. Phase 2 is now about 9 weeks.
4	Investigate on CPU-only boxes would exceed the 45 s limit; the 7 s trace is a GPU number.	Limits default by hardware class; Investigate is offered only when the site's model passes the harness; Look is offered everywhere.
5	Workflow webhooks from inside core would bypass the egress policy apps are held to.	Webhooks go only to admin-approved destinations through the same per-destination policy.
6	Workflow vision calls share KAI-C's per-camera budget with apps and could starve a plate read.	A workflow priority at KAI-C, so apps keep their budget.
7	Camera scope was not checked at save time.	The validator rejects cameras the saver cannot see.
8	Chain depth is lost when an app re-alerts on a workflow alert.	Stated; the hourly cap and cooldown contain it until the SDK passes the chain through in phase 3.
9	Unbounded site facts would crowd a small model's context.	Facts capped at about 300 words; longer material goes in lists.
10	The severity cap stops alarm actions but not notification channels.	Stated, with Test mode as the answer.
11	The `ask_person` tool's behaviour in Test was unstated.	Same as the step: records "would have asked", takes the default.

#	FINDING	CHANGE
12	Who may mark runs was unstated, though marks gate severity.	`workflows.manage` or `workflows.approve`.

Sources

- Research: what the best agentic workflow products do, with its product table and sources
- The original proposal: market, regulation and the case for workflows
- In the open-nvr repository: `examples/camera-agent/tools.py` (the ten tools), `docs/EVENT_CONTRACTS.md`, `docs/rfcs/RFC-0002-skills-and-apps.md`, `server/services/alerts_inbox.py`, `server/services/alarm_actions.py`, `server/services/camera_status_service.py`, `docker-compose.camera-agent.yml`